Callbacks

You order food at a restaurant.

You **don't stand inside the kitchen watching the chef**.
You give your order and say:

"When food is ready → call me."

That is a **callback**.

JavaScript works like this because:

- It **cannot block execution**
- It must continue handling:
    - clicks
    - API calls
    - timers
    - database responses

So we give it a function to call **later**.

A callback is a function passed as an argument to another function, which is executed later after a specific task or event is completed.

In JavaScript, callbacks are mainly used for:

- Handling **asynchronous operations** (API calls, timers, file reading)
- Executing code **after something finishes**
- Controlling **execution flow**

```
function greet(name, callback) {
   console.log("Hello " + name);
   callback();
}
```

```
function sayBye() {
   console.log("Goodbye!");
}

greet("Prajwal", sayBye);
```

Output:

Hello Prajwal

Goodbye!

Here:

- sayBye is passed → becomes a **callback**
- It runs **after** greet finishes.

⚠ Why Callbacks Became a Problem

Callback Hell

```
login(user, function() {
  getProfile(function() {
    getPosts(function() {
      getComments(function() {
        // 💀 unreadable
      });
    });
  });
});
```

Problems:

- Hard to debug
- No error propagation
- Pyramid structure
- Maintenance nightmare
-

Why Do We Need Callbacks in JavaScript?

Answer:
JavaScript is single-threaded and asynchronous. Callbacks allow us to execute code **after** a task finishes without blocking the main thread.

Are Callbacks Only Used for Asynchronous Code?

Answer:
No. Callbacks can be used in both synchronous and asynchronous code, but they are mainly useful in asynchronous operations.

What is Callback Hell?

Answer:
Callback Hell happens when multiple callbacks are nested inside each other, making code hard to read, debug, and maintain.

Also called **Pyramid of Doom**.

How Do You Avoid Callback Hell?

Answer:
We avoid it using:

- Promises
- Async/Await
- Modular functions

What is an Error-First Callback?

Answer:
In Node.js, callbacks follow the **error-first pattern** where the first parameter is an error object.

```
fs.readFile("file.txt", function(err, data){
  if(err){
    console.error(err);
    return;
  }
  console.log(data);
});
```

👉 Standard convention in Node.js.

What is a Higher-Order Function? How is it Related to Callbacks?
Answer:
A higher-order function is a function that accepts another function as an argument or returns a function.
Callbacks are used inside higher-order functions.

What Problems Do Callbacks Create?
Answer:

- Callback Hell
- Hard error handling
- Difficult debugging
- Inversion of control (we lose control over execution)

When Should You Use Callbacks Today?
Answer:
Callbacks are still useful for:

- Event handling (DOM events)
- Small async tasks
- Performance-critical code
- Streams in Node.js

But for complex flows → Promises/Async-Await are preferred.

Is setTimeout a Callback Example?
Answer:
Yes. The function passed into setTimeout is executed later, so it is a callback.

What is Inversion of Control in Callbacks?
Answer:
When we pass a callback, we give control of execution to another function, and we don’t know exactly when or how it will run.
This can create trust and debugging issues.

IN BRIEF
A callback is a function passed to another function to be executed later. It is mainly used in JavaScript to handle asynchronous, non-blocking operations.
While callbacks help manage execution flow, excessive nesting leads to callback hell, which is why modern JavaScript uses Promises and async/await for better readability and control.